

Generative AI — 20 Industry-Flavored Problem Statements

Based on: github.com/AnshulParate2004/Genrative_AI | Each core LangChain/LangGraph/RAG/MCP concept re-framed as a real industry use case

Industry Index

Industry	Problem #	Problem Statement Title(s)
Healthcare	4	RAG Pipeline with ChromaDB
Financial Services	10	Human-in-the-Loop Approval Workflow
Insurance	3, 20	Parallel & Conditional Chains; Nested Subgraph Multi-Agent
Banking	6	Chatbot with Persistent Memory (LangGraph)
Retail & E-Commerce	2, 19	Structured Output with Pydantic; Web Scraping Agent
Tourism	14, 18	Memory-Aware Chatbot with Mem0; TypeScript MCP Server (Weather)
Food & Beverages	16	Dynamic Prompt Templating & Dual-LLM Comparison
QSR (Quick Service Restaurants)	17	Persistent Chat History with Firebase
Telecommunications	12	Building a Custom MCP Server (File Manager)
Manufacturing	9	Supervisor Multi-Agent System
Automotive	13	Full-Stack MCP Application with FastAPI + Gemini
IT Services	1	LCEL Chain Building with LangChain
Supply Chain	7	ReAct Agent from Scratch
Aerospace	15	Math Reasoning Agent with Structured JSON Output
Medical Equipments	11	Comparing Text Chunking Strategies for RAG
Law Firms	5	Metadata-Filtered RAG
Consulting Services	8	Reflexion Agent (Self-Critique Loop)

1. LCEL Chain Building with LangChain

Industry: IT Services

Problem Statement

You're building an internal tool for a global IT services firm that documents client deliverables in English but needs them translated for offshore teams. Build an LCEL chain that takes a technical topic, generates 3 factual points about it, and translates those points into another language — all in one pipeline.

What they will learn

Prompt templates, output parsers, and how | (pipe) chaining works in LCEL to connect multiple steps.

Task and expected outcome

Given a topic input (e.g. "Kubernetes autoscaling"), output a French (or any language) translation of 3 facts about it — mirroring how an IT services team would auto-localize technical documentation. Outcome: a working .ipynb notebook with a 3-stage chain (facts → English → French).

Tools which can be used

Python, LangChain, Groq/Gemini API, ChatPromptTemplate, StrOutputParser.

Steps to follow

Set up API keys and install langchain, langchain-groq/langchain-google-genai. Create a prompt template for fact generation on a technical topic. Chain it with a translation prompt using LCEL |. Parse output as string and print.

Knowledge Test

What does LCEL stand for and why is it preferred over manual chaining? Difference between RunnableSequence and | syntax? What does StrOutputParser do?

Project submission

Submit chain.ipynb + a screenshot of console output + a short README explaining each stage.

2. Structured Output with Pydantic

Industry: Retail & E-Commerce

Problem Statement

An e-commerce marketplace receives thousands of unstructured product descriptions from third-party sellers. Create a LangChain pipeline that analyzes a raw product description and returns structured catalog data (title, category, price_tier, summary) instead of plain text.

What they will learn

How to enforce structured, type-safe LLM outputs using Pydantic models with LangChain.

Task and expected outcome

Input: a seller-submitted product description paragraph. Output: a validated JSON object matching a Pydantic schema, ready to drop straight into the product catalog database.

Tools which can be used

Python, LangChain, Pydantic, any chat LLM (Gemini/Groq).

Steps to follow

Define a Pydantic BaseModel with fields like title, category, price_tier. Bind it to the LLM using .with_structured_output(). Test with 3 different product descriptions.

Knowledge Test

Why use Pydantic instead of asking the LLM to "return JSON" in the prompt? What happens if the LLM output doesn't match the schema?

Project submission

Submit notebook (.ipynb) + 3 sample outputs (JSON) + explanation of schema fields.

3. Parallel & Conditional Chains

Industry: Insurance

Problem Statement

An insurance company wants to speed up claims processing. Build a chain that analyzes a claim's damage description and supporting evidence simultaneously (parallel: damage assessment + fraud-risk check), and a second chain that routes the claim to "auto-approve" or "manual review" conditionally based on the risk score.

What they will learn

Parallel execution (RunnableParallel) and conditional branching logic in LangChain.

Task and expected outcome

Two working scripts: one that runs 2 LLM calls in parallel (damage assessment + fraud check) and merges results; one that branches a claim to auto-approve or manual-review based on the combined risk output.

Tools which can be used

LangChain, RunnableParallel, RunnableBranch, any LLM API.

Steps to follow

Build parallel chain: define damage-assessment and fraud-risk prompts, combine with RunnableParallel. Build conditional chain: use the risk score to route to auto-approve or manual-review prompts. Test with sample claim descriptions.

Knowledge Test

When would you use parallel over conditional chains? How does RunnableBranch decide which path to take?

Project submission

Submit both notebooks (.ipynb) + example claims showing both branches triggered.

4. RAG Pipeline with ChromaDB

Industry: Healthcare

Problem Statement

A hospital wants clinicians to quickly query internal treatment protocols instead of digging through PDFs. Build a Retrieval-Augmented Generation (RAG) system that answers clinical questions from a custom protocol/guideline document using a vector database.

What they will learn

Embeddings, vector similarity search, and how retrieval improves LLM factual accuracy.

Task and expected outcome

Input: a clinical protocol document + a doctor's question. Output: an accurate, context-grounded answer with the source guideline passage shown — critical for clinical trust and auditability.

Tools which can be used

LangChain, ChromaDB, HuggingFace/Gemini embeddings, Python.

Steps to follow

Load and chunk a sample clinical guideline document. Generate embeddings and store in ChromaDB. Retrieve top-k relevant chunks for a clinical query. Pass retrieved context + query to the LLM for a final grounded answer.

Knowledge Test

Why is RAG better than fine-tuning for factual QA? What is cosine similarity used for here?

Project submission

Submit notebook (.ipynb) + a sample document + 3 Q&A; pairs showing retrieved sources.

5. Metadata-Filtered RAG

Industry: Law Firms

Problem Statement

A law firm's associates need to search case law without wading through irrelevant jurisdictions. Extend a basic RAG system to filter retrieval results using metadata (e.g., only search precedents from a specific jurisdiction/year/court).

What they will learn

Metadata tagging during embedding and filtered vector search for more precise retrieval.

Task and expected outcome

A legal RAG system where a query like "what did the Bombay High Court say about X in 2023" only searches chunks tagged jurisdiction: bombay_hc, year: 2023.

Tools which can be used

LangChain, ChromaDB/Qdrant, Python.

Steps to follow

Tag each case-law chunk with metadata (jurisdiction, year, court) during embedding. Use metadata filters in the retriever query. Compare results with vs. without filtering.

Knowledge Test

What's the benefit of metadata filtering over pure semantic search? Give an example use case where this is essential.

Project submission

Submit notebook (.ipynb) + before/after comparison of filtered vs unfiltered retrieval results.

6. Chatbot with Persistent Memory (LangGraph)

Industry: Banking

Problem Statement

A bank's customer support chatbot keeps forgetting a customer's loan case details between messages, frustrating users. Build a chatbot using LangGraph that remembers conversation history across multiple turns and sessions using SQLite checkpointing.

What they will learn

LangGraph's StateGraph, checkpointing, and how state persists across a conversation.

Task and expected outcome

A CLI banking support chatbot that recalls earlier messages (e.g. loan application ID, prior questions) in the same session, and resumes correctly after a restart — no repeating themselves for the customer.

Tools which can be used

LangGraph, SQLite, Python, any LLM API.

Steps to follow

Define chatbot state schema. Build a single-node graph with a chatbot function. Add SqliteSaver checkpointing. Test memory persistence by restarting the script mid-conversation.

Knowledge Test

What is a "checkpoint" in LangGraph? How is this different from just storing chat history in a list?

Project submission

Submit notebook (.ipynb) + a demo transcript showing memory recall after restart.

7. ReAct Agent from Scratch

Industry: Supply Chain

Problem Statement

A logistics company wants an assistant that can answer live shipment questions. Implement a ReAct (Reason + Act) agent using LangGraph that answers questions by reasoning step-by-step and calling an external tool (e.g., a shipment-tracking API).

What they will learn

The ReAct pattern: how an agent alternates between reasoning, acting (tool calls), and observing results.

Task and expected outcome

Given a question like "Where is shipment #4521 right now and will it arrive on time?", the agent reasons, calls the tracking API tool, observes the result, and gives a final answer.

Tools which can be used

LangGraph, Python, a public REST API (e.g. a tracking/logistics API), requests.

Steps to follow

Define agent state (question, thoughts, actions, observations). Build reasoning node and tool-execution node. Loop reason→act→observe until final answer. Test with 2-3 different shipment questions.

Knowledge Test

What are the 3 phases of the ReAct loop? Why is observation important before the next reasoning step?

Project submission

Submit notebook (.ipynb) + full trace log of one complete reason-act-observe cycle.

8. Reflexion Agent (Self-Critique Loop)

Industry: Consulting Services

Problem Statement

A consulting firm wants first-draft client deliverables to be sharper before a human reviews them. Build an agent that answers a client question/brief, critiques its own answer, and revises it — repeating until a quality threshold or max iterations is reached.

What they will learn

Self-reflection patterns in agentic AI: how an LLM can evaluate and improve its own output.

Task and expected outcome

Given a consulting-style question (e.g. "outline a market-entry strategy for X"), the agent produces an initial answer, generates a critique, revises the answer, and stops after N iterations or when critique says "good enough."

Tools which can be used

LangGraph, Pydantic (for structured critique schema), Python.

Steps to follow

Define schema for "responder" and "reviser" outputs. Build first-responder chain and revision chain. Add a loop with MAX_ITERATIONS control. Log each iteration's answer and critique.

Knowledge Test

What risk exists if there's no MAX_ITERATIONS limit? How does self-critique differ from human-in-the-loop review?

Project submission

Submit notebook (.ipynb) + log showing at least 2 iterations of critique-and-revise.

9. Supervisor Multi-Agent System

Industry: Manufacturing

Problem Statement

A manufacturing plant wants one assistant that can triage production issues end-to-end. Build a hierarchical multi-agent system where a "Supervisor" agent routes requests to specialized agents (Researcher, Coder, Enhancer) and validates the final output.

What they will learn

Multi-agent orchestration, dynamic routing, and how specialist agents can collaborate under a supervisor.

Task and expected outcome

Given a mixed query (e.g., "research causes of downtime on Line 3 and write code to analyze the sensor logs"), the system routes correctly between agents and returns a validated final response.

Tools which can be used

LangGraph, Pydantic, Tavily Search API, Python REPL tool.

Steps to follow

Define Supervisor, Enhancer, Researcher, Coder, Validator nodes. Add routing logic based on structured output from Supervisor. Connect Researcher to a web search tool and Coder to a Python REPL for sensor-log analysis. Add a Validator node to check output quality and loop back if needed.

Knowledge Test

Why use a Supervisor pattern instead of one large agent? What does the Validator node protect against?

Project submission

Submit notebook (.ipynb) + architecture diagram + one full run log (query → routing → final answer).

10. Human-in-the-Loop Approval Workflow

Industry: Financial Services

Problem Statement

A trading desk needs an AI assistant that can prep trades but must never execute one without a human sign-off. Build a LangGraph workflow that pauses execution and waits for human approval before proceeding with a sensitive action (e.g., executing a trade or sending a client email).

What they will learn

Interrupt/resume patterns in LangGraph and how to build safe agentic workflows with manual checkpoints.

Task and expected outcome

A graph that stops at a decision node, asks for human input (approve/reject) before a trade or fund transfer proceeds, and continues or halts accordingly.

Tools which can be used

LangGraph (interrupt, Command), Python.

Steps to follow

Design a graph with a "trade execution" node requiring approval. Use LangGraph's interrupt mechanism to pause execution. Accept user input to resume (resume.py) or reject. Test both approve and reject paths.

Knowledge Test

Why is human-in-the-loop important for agentic systems? What's the difference between interrupt and simply pausing with input()?

Project submission

Submit notebook (.ipynb) + two demo runs (one approved, one rejected).

11. Comparing Text Chunking Strategies for RAG

Industry: Medical Equipments

Problem Statement

A medical device manufacturer's support team needs fast, accurate answers from thick equipment manuals (IFUs). Implement and compare at least 4 different text chunking strategies (fixed-size, recursive, sliding window, semantic) on the same manual, and measure their effect on RAG answer quality.

What they will learn

How chunking strategy affects retrieval precision and downstream answer accuracy in RAG systems.

Task and expected outcome

A report/notebook comparing retrieval results and answer quality across chunking methods for the same set of test questions on a device manual — where getting the wrong answer has real safety implications.

Tools which can be used

LangChain text splitters, ChromaDB, Python, Jupyter Notebook.

Steps to follow

Load a sample document (e.g. a device instructions-for-use manual). Apply 4 chunking strategies separately. Build a RAG pipeline for each and run the same 5 test questions. Compare answer accuracy and relevance.

Knowledge Test

Why might semantic chunking outperform fixed-size chunking? What's the trade-off of smaller vs larger chunk sizes?

Project submission

Submit notebook with comparison table + a short written conclusion.

12. Building a Custom MCP Server (File Manager)

Industry: Telecommunications

Problem Statement

A telecom NOC team wants an LLM assistant that can safely read and edit network configuration files. Build a Model Context Protocol (MCP) server that exposes file operations (read, write, edit, search) as tools an LLM client can call.

What they will learn

MCP architecture (client-server), how to define and expose custom tools to an LLM.

Task and expected outcome

A working MCP server that an LLM client (e.g. Claude Desktop or a custom client) can connect to and use to read/edit network config files on request.

Tools which can be used

Python mcp SDK, FastAPI (optional), UV package manager.

Steps to follow

Install the MCP Python SDK. Define tool functions (read_file, write_file, find_replace, list_dir) scoped to a config directory. Register tools with the MCP server using decorators. Connect a client and test each tool call.

Knowledge Test

What problem does MCP solve compared to hardcoding tool integrations? What is the role of stdio transport in MCP servers?

Project submission

Submit server code (as .ipynb where applicable) + a short demo video/log of the client calling at least 2 tools.

13. Full-Stack MCP Application with FastAPI + Gemini

Industry: Automotive

Problem Statement

An automotive dealer network wants a single API that lets service advisors query vehicle diagnostics via chat. Build a full-stack application where a FastAPI backend connects an MCP client to an MCP server and exposes a /query endpoint powered by Gemini.

What they will learn

End-to-end integration of LLMs, MCP tool servers, and REST APIs into a deployable application.

Task and expected outcome

A working API where a POST request to /query (e.g. "why is the check-engine light on for VIN X") triggers Gemini to reason and call MCP diagnostic tools as needed, returning a final structured response.

Tools which can be used

FastAPI, MCP SDK, Gemini API, Python, CORS middleware.

Steps to follow

Build an MCP server with 2-3 simple diagnostic-lookup tools. Build an MCP client that discovers and calls tools. Wrap the client in a FastAPI /query and /tools endpoint. Add CORS config and test with a frontend or Postman.

Knowledge Test

What is the flow of a request from frontend → FastAPI → MCP client → MCP server → LLM? Why is async lifecycle management important here?

Project submission

Submit backend code (as .ipynb where applicable) + Postman collection/screenshots showing /tools and /query working.

14. Memory-Aware Chatbot with Mem0 + Vector DB

Industry: Tourism

Problem Statement

A travel booking platform wants its chat concierge to remember a traveler's preferences across trips. Build a chatbot with hybrid memory (short-term conversation + long-term vector-searchable memory) using Mem0 and a vector database.

What they will learn

How long-term memory retrieval works in agents, and how to combine it with short-term conversational context.

Task and expected outcome

A travel-concierge chatbot that remembers preferences mentioned in earlier sessions (e.g. "I prefer window seats and vegetarian meals") and recalls them naturally when planning a later trip.

Tools which can be used

Mem0, Qdrant/ChromaDB, Groq/Gemini, HuggingFace embeddings.

Steps to follow

Set up Mem0 with a vector DB backend. On each user message, store relevant travel preferences to long-term memory. On new queries, retrieve relevant memories and inject into the prompt. Add basic rate-limit/backoff handling for API calls.

Knowledge Test

What's the difference between short-term and long-term memory in an agent? Why use a vector DB instead of a simple key-value store for memory?

Project submission

Submit notebook (.ipynb) + a 2-session demo showing the bot recalling a preference from session 1 in session 2.

15. Math Reasoning Agent with Structured JSON Output

Industry: Aerospace

Problem Statement

A flight-ops team needs quick, auditable fuel and load calculations. Build an agent that solves step-by-step quantitative problems (e.g. fuel burn, payload/balance) using a reasoning-focused LLM and outputs the solution as structured JSON (steps + final answer).

What they will learn

Using reasoning-optimized models for multi-step problem solving and enforcing structured, parseable output.

Task and expected outcome

Given a fuel/load calculation word problem, output a JSON object with an array of solution steps and a final numeric answer, saved to a file for the flight log.

Tools which can be used

Groq API (DeepSeek-R1-Distill-Llama-70B or similar), Python, json module.

Steps to follow

Design a prompt requiring step-by-step reasoning in JSON format. Call the reasoning model with 5 different fuel/load problems. Parse and validate JSON output. Save each solution to a .json file.

Knowledge Test

Why are reasoning-distilled models better suited for this task than general chat models? What could go wrong if the model doesn't strictly follow the JSON schema, and how would you handle it?

Project submission

Submit notebook (.ipynb) + 5 solved problems saved as JSON files + one screenshot of console output.

16. Dynamic Prompt Templating & Dual-LLM Comparison

Industry: Food & Beverages

Problem Statement

A food brand's R&D team wants to compare how two different LLMs answer nutrition/recipe questions before picking one for their consumer app. Build a chatbot that sends the same user message to two different LLMs (e.g., Gemini and Groq Llama-3.3-70B) using dynamic prompt templates, and displays both responses side by

side.

What they will learn

How to build reusable prompt templates with system + human messages, and how to swap/compare LLM backends without changing the prompt logic.

Task and expected outcome

Given a nutrition/recipe query, the notebook outputs both models' responses in a clearly labeled, side-by-side format, with persistent chat history across turns.

Tools which can be used

LangChain, ChatPromptTemplate, Gemini API, Groq API, Python.

Steps to follow

Set up API keys for both Gemini and Groq. Build one shared ChatPromptTemplate with system + human message slots. Call both models with the same formatted prompt. Store and display alternating responses with chat history.

Knowledge Test

Why use one shared template for two models instead of separate prompts? What differences might you expect between Gemini and Groq Llama responses, and why?

Project submission

Submit notebook (.ipynb) + a transcript showing 3 user turns with both models' responses.

17. Persistent Chat History with Firebase

Industry: QSR (Quick Service Restaurants)

Problem Statement

A QSR chain's order-assistant chatbot should remember a regular customer's usual order across visits and devices. Build a chatbot that stores and retrieves conversation history from Firebase Firestore.

What they will learn

Cloud-based persistence for conversational AI, and how to structure conversation data in a NoSQL document database.

Task and expected outcome

A QSR ordering chatbot where closing and reopening the app still recalls the full prior order conversation, loaded from Firestore.

Tools which can be used

Firebase Firestore, firebase-admin SDK, LangChain, Python.

Steps to follow

Set up a Firebase project and service account credentials. Write functions to save each message (role + content) to a Firestore collection keyed by customer ID. Write a function to load chat history at session start. Test by restarting the notebook and confirming history loads.

Knowledge Test

Why choose Firestore over local storage (e.g., SQLite) for chat history? What data model would you use to store messages per user/session?

Project submission

Submit notebook (.ipynb) + Firestore console screenshot showing saved messages.

18. TypeScript MCP Server (Weather Data)

Industry: Tourism

Problem Statement

A trip-planning assistant needs live weather to warn travelers about their destination. Build a Model Context Protocol server in TypeScript/Node.js that exposes a "get weather by city" tool, using stdio transport and schema validation.

What they will learn

How MCP works outside Python — server setup in Node.js, schema validation with Zod, and stdio-based client-server communication.

Task and expected outcome

A working MCP server that returns weather data for a traveler's destination city when called by an MCP client (e.g. Claude Desktop), feeding straight into a trip-planning workflow.

Tools which can be used

Node.js, @modelcontextprotocol/sdk, Zod, TypeScript, a public weather API.

Steps to follow

Set up a Node.js/TypeScript project with the MCP SDK. Define a Zod schema for the tool's input (city name). Implement the tool function calling a weather API. Register the server with stdio transport and test with an MCP client.

Knowledge Test

Why is schema validation (Zod) important for tool inputs in MCP? How does stdio transport differ from an HTTP-based MCP server?

Project submission

Submit server code + a demo log/screenshot of a successful tool call from an MCP client.

19. Web Scraping Agent with Search API Integration

Industry: Retail & E-Commerce

Problem Statement

A retail brand's pricing team wants to track competitor pricing and promotions automatically. Build an agent that takes a research query, searches the web using the Serper API, scrapes the top results, and summarizes findings from multiple sources.

What they will learn

How to combine a search API with web scraping and LLM summarization to build a lightweight research agent.

Task and expected outcome

Given a query like "latest discount trends from competing retailers this quarter," the notebook returns a summarized report combining data from at least 3 scraped sources.

Tools which can be used

Serper API, requests/BeautifulSoup, LangChain, Python.

Steps to follow

Call the Serper API to get top search results for a competitor-pricing query. Scrape and clean text content from each result URL. Pass combined content to an LLM for summarization. Output a structured summary with source links.

Knowledge Test

Why use a search API instead of scraping search engine pages directly? What are the risks/limitations of scraping arbitrary websites?

Project submission

Submit notebook (.ipynb) + one sample query with the final summarized report and source links.

20. Nested Subgraph Multi-Agent Architecture

Industry: Insurance

Problem Statement

An insurer wants an end-to-end claims-processing assistant: one outer flow that manages the whole claim lifecycle, and an inner flow that specifically handles damage assessment. Build a LangGraph system where a top-level supervisor graph calls a nested subgraph (its own internal multi-step workflow) as a single node.

What they will learn

How to compose complex agent systems by nesting one StateGraph inside another, and why modular subgraphs improve maintainability.

Task and expected outcome

A working two-level graph: an outer claims-supervisor routes to a damage-assessment subgraph node, which internally runs its own multi-step process (research photos → estimate cost → summarize) before returning a result to the outer graph.

Tools which can be used

LangGraph, Python, any LLM API.

Steps to follow

Build an inner subgraph with 2-3 nodes performing damage research + cost estimation + summarization. Compile the subgraph and wrap it as a callable node in the outer graph. Build the outer supervisor graph that routes to the subgraph and other claim-handling nodes. Test the full flow end-to-end and log state at each level.

Knowledge Test

What's the benefit of a subgraph over just adding more nodes to one big graph? How does state get passed between the outer graph and the subgraph?

Project submission

Submit notebook (.ipynb) + a diagram of the outer graph + subgraph relationship + one full run log.

End of document — 20 industry-flavored problem statements covering LangChain, LangGraph, RAG, multi-agent systems, MCP servers (Python & TypeScript), and memory architectures across Healthcare, Financial Services, Insurance, Banking, Retail & E-Commerce, Tourism, Food & Beverages, QSR, Telecommunications, Manufacturing, Automotive, IT Services, Supply Chain, Aerospace, Medical Equipments, Law Firms, and Consulting Services.